

# Module : 7 : React - Fundamentals

**Q - 1 : What is React.js? How is it different from other JavaScript frameworks and libraries?**

A -1 : React (or React.js) is a popular, open-source JavaScript library used for building user interfaces (UIs), specifically for dynamic and interactive web applications.

| Feature        | React.js                                                         | Angular                                                  | Vue.js                                                       |
|----------------|------------------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------|
| Type           | Library                                                          | Framework                                                | Progressive Framework                                        |
| Flexibility    | High; developers choose their own tools for routing, state, etc. | Low; comes with built-in tools (opinionated)             | Moderate; offers a balance between flexibility and structure |
| DOM Usage      | Virtual DOM                                                      | Regular DOM (though highly optimized)                    | Virtual DOM                                                  |
| Learning Curve | Moderate                                                         | Steep; requires TypeScript and complex built-in concepts | Low; very beginner-friendly with a gentle curve              |
| Data Binding   | One-way                                                          | Two-way                                                  | Two-way                                                      |

**Q - 2 : Explain the core principles of React such as the virtual DOM and component-based architecture.**

A - 2 : The core principles of React center on enhancing application performance and simplifying development through Component-Based Architecture and the Virtual DOM.

## 1. Component-Based Architecture

- React builds user interfaces by breaking them down into small, isolated, and reusable pieces called components.
  - Modularity
  - Reusability
  - Composition
  - Predictable Data Flow

## 2. The Virtual DOM

- The Virtual DOM (VDOM) is a lightweight, in-memory representation of the real browser DOM.

How the Virtual DOM Works:

- State Change
- Diffing
- Reconciliation
- Batch Updates

### **Q - 3 : What are the advantages of using React.js in web development?**

A - 3 : React.js is widely used in web development primarily for its component-based architecture, high performance via the Virtual DOM, and vast ecosystem.

- It allows developers to build complex user interfaces (UIs) more efficiently by breaking them into manageable, reusable pieces.

### **Key Advantages of React.js :**

- Component Reusability
- Virtual DOM for Performance
- SEO-Friendly Rendering
- Declarative UI
- Vast Ecosystem and Community
- Cross-Platform Capability
- JSX for Readable Code

### **Q - 4 : What is JSX in React.js? Why is it used?**

A - 4 : JSX (JavaScript XML) is a syntax extension for JavaScript that allows you to write HTML-like code directly inside your React components.

- While it looks like markup, it is actually powerful JavaScript that describes what the UI should look like.

### **It is Used for :**

- Improved Readability
- Declarative UI

- Dynamic Logic Integration
- Security
- Easier Maintenance

**Q - 5 : How is JSX different from regular JavaScript? Can you write JavaScript inside JSX?**

**A - 5 : JSX different from regular JavaScript with :**

- **Syntax:** JSX allows you to write HTML-like tags (e.g., `<h1>Hello</h1>`) directly within your code, whereas regular JavaScript requires manual DOM manipulation or methods like `document.createElement()` to achieve the same result.
- **Compilation:** Standard JavaScript can be run directly by browsers, but JSX is not valid JavaScript.
- **Declarative Nature:** JSX is more declarative, focusing on what the UI should look like, while regular JavaScript often focuses on the how (the imperative steps to update the UI)

**Writing JavaScript Inside JSX :**

- Yes, you can write JavaScript inside JSX by using curly braces `{ }`.
- These braces act as a "window" back into regular JavaScript, allowing you to embed:

**Q - 6 : Discuss the importance of using curly braces {} in JSX expressions**

**A - 6 :** In JSX, curly braces `{ }` are important because they allow you to write JavaScript expressions inside HTML-like code.

**Importance of Curly Braces {} in JSX:**

1. **Display Variables**
  - Curly braces are used to show variable values in JSX.
2. **Use JavaScript Expressions**
  - You can perform calculations or expressions inside `{}`.
3. **Conditional Rendering**
  - Used for conditions like if, ternary operator, etc
4. **Dynamic Content**
  - Helps render dynamic data.

**Q - 7 : What are components in React? Explain the difference between functional components and class components.**

A - 7 : Components are the building blocks of React applications.

- They are reusable pieces of code used to create different parts of a user interface (UI), such as buttons, forms, headers, and pages.

**Difference Between Functional Components and Class Components :**

| Functional Components                                       | Class Components                     |
|-------------------------------------------------------------|--------------------------------------|
| Written as JavaScript functions                             | Written using ES6 classes            |
| Simpler and easier to write                                 | More complex syntax                  |
| Use <b>Hooks</b> (useState, useEffect) for state management | Use this.state and lifecycle methods |
| Faster and commonly used in modern React                    | Older approach, less used now        |
| No render() method needed                                   | Requires render() method             |

**Q - 8 : How do you pass data to a component using props?**

A - 8 : In React, props (properties) are used to pass data from a parent component to a child component.

- We pass data as attributes in the component and receive it using props.

| Parent Component                                                                                 | Child Component                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>function App() {<br/>  return &lt;Student name="Vrutti"<br/>    age="20" /&gt;;<br/>}</pre> | <pre>function Student(props) {<br/>  return (<br/>    &lt;h1&gt;<br/>      Name: {props.name}, Age:<br/>      {props.age}<br/>    &lt;/h1&gt;<br/>  );<br/>}</pre> |

**Q - 9 : What is the role of render() in class components?**

A - 9 : In React class components, the render() method is used to display JSX (UI) on the screen.

- It is a required method in every class component because React calls it to decide what should be shown in the browser.

**Q - 10 : What are props in React.js? How are props different from state?**

A - 10 : Props (Properties) are used in React to pass data from a parent component to a child component. Props make components reusable and dynamic

**Difference Between Props and State :**

| Props                                     | State                                    |
|-------------------------------------------|------------------------------------------|
| Used to pass data from parent to child    | Used to manage data within a component   |
| Read-only (cannot be changed)             | Can be changed/updated                   |
| Passed as arguments                       | Managed inside the component             |
| Used for communication between components | Used for dynamic behavior and UI updates |

**Q - 11 : Explain the concept of state in React and how it is used to manage component data.**

A - 11 : State in React is an object used to store and manage data inside a component. It allows components to change and update dynamically when data changes.

- State is used for data that can change over time, such as counters, form inputs, login status, etc.

**How State Manages Component Data:**

1. Stores dynamic data inside a component.
2. Updates UI automatically when data changes.
3. Makes components interactive and dynamic.

**Q - 12 : Why is this.setState() used in class components, and how does it work?**

A - 12 : In React class components, this.setState() is used to update the state (data) of a component.

- We use this.setState() because state should not be changed directly. React uses it to update the UI automatically when state changes.

**How this.setState() Works:**

1. It updates the state value.
2. React re-renders the component automatically.
3. The UI shows the updated data.

**Q - 13 : What is conditional rendering in React? How can you conditionally render elements in a React component?**

A - 13 : Conditional rendering in React means displaying different UI elements based on a condition.

- It works like JavaScript conditions (if, else, ternary operator, &&) to show or hide content.

- **if-else** → Used for multiple conditions.
- **Ternary operator** → Used for simple conditions.
- **&& operator** → Used to show something only if the condition is true.

**Q - 14 : Explain how if-else, ternary operators, and &&(logical AND) are used in JSX for conditional rendering.**

A - 14 :

**1. Using if-else :**

- if-else is used when there are **multiple conditions**.

```
function App() {  
  const isLoggedIn = true;  
  
  if (isLoggedIn) {  
    return <h1>Welcome User</h1>;  
  } else {  
    return <h1>Please Login</h1>;  
  }  
}
```

## 2. Using Ternary Operators (? :) :

- The **ternary operator** is used for **simple conditions** inside JSX.

```
function App() {  
  const isLoggedIn = false;  
  
  return (  
    <h1>  
      {isLoggedIn ? "Welcome User" : "Please Login"}  
    </h1>  
  );  
}
```

## 3. Using Logical AND (&&) :

- The **&& operator** is used when you want to **show something only if the condition is true**.

```
function App() {  
  const isAdmin = true;  
  
  return (  
    <div>  
      {isAdmin && <h1>Admin Panel</h1>}  
    </div>  
  );  
}
```

**Q - 15 : How do you render a list of items in React? Why is it important to use keys when rendering lists?**

**A - 15 :** In React, a list of items is rendered using the **map() method**.

- It helps display multiple elements dynamically.
- **A key is a special attribute used to uniquely identify each item in a list.**

### **Importance of Keys :**

1. Helps React identify items uniquely.
2. Improves performance by updating only changed items.
3. Prevents rendering errors when items are added, removed, or updated.

### **Q - 16 : What are keys in React, and what happens if you do not provide a unique key?**

A - 16 :

- **Keys** in React are special attributes used to **uniquely identify elements in a list**.
- They help React track which items have changed, been added, or removed.
- Keys are mainly used when rendering lists with the `map()` method.

### **What happens if you do not provide a unique key?**

1. React shows a warning message in the console.
2. React may update the wrong elements.
3. It can cause performance issues and unexpected UI behavior.

### **Q - 17 : How do you handle forms in React? Explain the concept of controlled components.**

A - 17 : In React, forms are handled using state to manage input values.

- React uses event handlers like `onChange` and `onSubmit` to handle form data.
- A **controlled component** is a form element whose value is **controlled by React state**.
  - This means React controls the form data instead of the browser.
- **Why Controlled Components are Important:**
  1. Form data is managed by **React state**.
  2. Easy to **validate and control input**.
  3. Makes forms **predictable and manageable**.

### **Q - 18 : What is the difference between controlled and uncontrolled components in React?**

A - 18 :



| Controlled Components                       | Uncontrolled Components                        |
|---------------------------------------------|------------------------------------------------|
| Data is controlled by <b>React state</b>    | Data is controlled by the <b>DOM (browser)</b> |
| Uses <b>useState()</b> or <b>this.state</b> | Uses <b>ref</b> to access values               |
| Easier to validate and manage               | Less control over form data                    |
| React controls input value                  | Browser controls input value                   |

**Q - 19 : What are lifecycle methods in React class components? Describe the phases of a component's lifecycle.**

A - 19 : Lifecycle methods are special methods in React class components that are called at different stages of a component's life, such as **creation, updating, and removal**.

- They allow developers to control what happens when a component **mounts, updates, or unmounts**.

**Phases of a Component's Lifecycle :**

**1. Mounting Phase :**

- This phase occurs when the component is created and added to the DOM.

**Common methods:**

- constructor() → Initializes state and props.
- render() → Displays UI.
- componentDidMount() → Runs after component is rendered.

**2. Updating Phase**

- This phase occurs when **state or props change** and the component re-renders.

**Common methods:**

- shouldComponentUpdate() → Decides if a component should update.
- render() → Re-renders UI.
- componentDidUpdate() → Runs after update.

### 3. Unmounting Phase :

- This phase occurs when the component is **removed from the DOM**.

#### Method:

- `componentWillUnmount()` → Used for cleanup (timers, API calls, etc.)

**Q - 20 : Explain the purpose of `componentDidMount()`, `componentDidUpdate()`, and `componentWillUnmount()`.**

A - 20 :

**1. `componentDidMount()` :** This method is called **after the component is rendered (mounted) to the DOM**.

#### Purpose:

- Fetch API data
- Start timers
- Perform setup tasks

#### Example :

```
class App extends React.Component {  
  componentDidMount() {  
    console.log("Component Mounted");  
  }  
  
  render() {  
    return <h1>Hello World</h1>;  
  }  
}
```

**Explanation:** It runs only once when the component loads

**2. `componentDidUpdate()` :** This method is called **after the component updates** due to changes in **state or props**.

#### Purpose:

- Update data after state changes
- Perform actions when component updates

**Example :**

```
componentDidUpdate() {  
  console.log("Component Updated");  
}
```

**Explanation :** It runs whenever the component is re-rendered after an update.

**3. componentWillUnmount() :** This method is called **before the component is removed from the DOM.**

**Purpose:**

- Clean up timers
- Cancel API calls
- Remove event listeners

**Example :**

```
componentWillUnmount() {  
  console.log("Component Removed");  
}
```

**Explanation:** It helps prevent memory leaks by cleaning resources.